



Delft University of Technology

Delft University of Technology
Faculty of Electrical Engineering, Mathematics and Computer Science
Department of Electrical and Sustainable Energy
Group of DC Systems, Energy Conversion and Storage

User Manual

MCES_library.py

Author:
Dr. Joel Alpízar-Castillo

Version 1.0
31 July, 2025

Contents

1	Introduction	1
2	csvreader	2
2.1	read_data Class	2
3	MCES_library	4
3.1	T_sky()	4
3.2	Wind_forced_convection()	4
3.3	radiative_heat_transfer_coefficient()	5
3.4	Thermal_diffusivity()	5
3.5	radiative_heat_transfer_coefficient_two_plates()	6
3.6	conductive_heat_transfer_coefficient()	6
3.7	Rayleigh()	6
3.8	Nusselt()	7
3.9	Reynolds()	7
3.10	Prandlt()	8
3.11	fluid_conductive_heat_transfer()	8
3.12	air_gap_convection()	9
3.13	PVT_model()	9
3.14	Qdot_PVT()	11
3.15	T_fluid_out()	12
3.16	PV_efficiency()	12
3.17	PV_out()	12
3.18	Update_PVT_tank()	13
3.19	Heat_exchanger()	13
3.20	Qdot_SolarCollector()	14
3.21	update_TESS()	14
3.22	Qdot_SD_TESS2Soil()	15
3.23	HP_Power_0()	16
3.24	HP_Power()	16
3.25	BESS_perm_min()	17
3.26	BESS_perm_max()	17
3.27	update_BESS()	17
3.28	heat_loss()	18
3.29	new_house_Temperature()	19
3.30	R_convective()	19
3.31	R_conductive()	20

3.32	Celcius2Fahrenheit()	20
3.33	Infiltration_Airflow()	20
3.34	Ventilation_Airflow()	21
3.35	Infiltration_Ventilation_Heat_loss()	21
3.36	pipe_losses()	22
3.37	House_type_adjust()	23
3.38	Windows_losses_coefficient()	23
3.39	Roof_losses_coefficient()	24
3.40	Wall_losses_coefficient()	24
3.41	House_Thermal_Losses()	25
3.42	Thermal_Electrical_model()	26
3.43	Thermal_model()	28

Chapter 1

Introduction

This document provides a description of the functions used to simulate the residential multi-carrier energy systems, together with a low-voltage distribution networks in the dissertation [1]. Each chapter provides a description of each of the functions used, including a brief mathematical explanation and details of the arguments. In addition, example files are also provided.

Note that all the functions and example files were tested in Synder 5.5.1, using Python 3.12.7, and Windows 10.

The FLEXINet project was carried out with a Top Sector Energy subsidy from the Ministry of Economic Affairs and Climate, carried out by the Netherlands Enterprise Agency (RVO). The specific subsidy for this project concerns the MOOI subsidy round 2020.

Chapter 2

csvreader

This library was not created for the FLEXINet project, but as part of the MSc thesis [2]. Nevertheless, as it is constantly used, its functions are also detailed. The library itself is stored in the repository in [3].

For this library to be used, the following module must be installed:

- `itertools`
- `pandas`
- `numpy`

2.1 `read_data` Class

The `read_data` class provides a lightweight interface for reading and processing CSV files without relying on full-featured data frames. It allows conversion of CSV data into lists of columns, rows, matrices, or arrays.

Constructor

```
read_data(csv='', delim=';', address='CSVs\\')
```

Arguments:

- **csv** [str]: Name of the CSV file, including extension (e.g., "`data.csv`").
- **delim** [str, optional]: Delimiter used in the CSV file (default is `'`; `'`).
- **address** [str, optional]: Path to the folder containing the CSV file (default is `'CSVs'`).

Attributes:

- **df**: The raw data frame read from the CSV.
- **nrows**: Number of rows in the data.
- **ncols**: Number of columns in the data.

Methods

data2cols()

Converts the data frame into a list of columns. Each column is stored as a list in `self.col`.

data2rows()

Converts the data frame into a list of rows. Each row is stored as a list in `self.row`.

data2matrix()

Converts the data frame into a NumPy matrix and stores it in `self.mat`.

data2array()

Converts the data frame into a NumPy array and stores it in `self.ar`.

Chapter 3

MCES_library

This library contains the functions used to simulate, individually and together, the components of a residential multi-carrier energy system, as described in [4]. The library itself is stored in the repository in [5].

For this library to be used, the following modules must be installed:

- numpy
- math
- scipy

3.1 T_sky()

This function estimates the effective sky temperature (in degrees Celsius) based on ambient temperature and relative humidity. It is used in radiative heat transfer calculations for surfaces exposed to the sky.

Formula:

$$T_{\text{sky}} = 0.62 + 0.056 \left[6.11 \cdot \frac{\text{RH}}{100} \cdot \exp \left(\frac{17.63 \cdot T_{\text{amb}}}{T_{\text{amb}} + 243} \right) \right]^{0.5} \quad (3.1)$$

Arguments:

- **T_amb [float]:** Ambient temperature in degrees Celsius.
- **RH [float, optional]:** Relative humidity in percentage (default is 70%).
- **N [float, optional]:** Cloud cover index (default is 0). This parameter is not used in the current implementation.

3.2 Wind_forced_convection()

This function estimates the convective heat transfer coefficient due to wind over a surface. It uses the McAdams correlation, which is suitable for low to moderate wind speeds.

Formula:

$$h = \begin{cases} 5.7 + 3.8 \cdot v, & \text{if } v < 5 \\ 6.47 + v^{0.78}, & \text{if } v \geq 5 \end{cases} \quad (3.2)$$

Notes:

- This function is based on empirical correlations and is commonly used in solar energy and building physics applications.
- The McAdams correlation is valid for wind speeds typically encountered in outdoor environments.

Arguments:

- **v [float]:** Wind speed in meters per second (m/s).

3.3 radiative_heat_transfer_coefficient()

This function calculates the radiative heat transfer coefficient between a surface and the sky, based on the Stefan-Boltzmann law. It is used to model longwave radiation exchange in thermal systems.

Formula:

$$h_r = \varepsilon \cdot \sigma \cdot [(T_s + 273.15)^2 + T_{\text{sky}}^2] \cdot [(T_s + 273.15) + T_{\text{sky}}] \quad (3.3)$$

where σ is the Stefan-Boltzmann constant and T_{sky} is estimated using the `T_sky()` function.

Notes:

- Temperatures are internally converted to kelvin.
- This coefficient is useful for estimating radiative losses from surfaces like solar collectors or building envelopes.

Arguments:

- **T_amb [float]:** Ambient air temperature in degrees Celsius.
- **T_s [float]:** Surface temperature in degrees Celsius.
- **epsilon [float]:** Emissivity of the surface (dimensionless, between 0 and 1).

3.4 Thermal_diffusivity()

This function calculates the thermal diffusivity of a material, which is a measure of how quickly heat spreads through it. Thermal diffusivity is important in transient heat conduction analysis.

Formula:

$$\alpha = \frac{k}{\rho \cdot c_p} \quad (3.4)$$

Arguments:

- **k [float]:** Thermal conductivity of the material in W/(m·K).
- **rho [float]:** Density of the material in kg/m³.
- **cp [float]:** Specific heat capacity of the material in J/(kg·K).

3.5 radiative_heat_transfer_coefficient_two_plates()

This function calculates the radiative heat transfer coefficient between two parallel plates with different temperatures and emissivities, based on the Stefan-Boltzmann law. It is used to model longwave radiation exchange between surfaces in thermal systems.

Formula:

$$h_{r,12} = \frac{\sigma [(T_1 + 273.15)^2 + (T_2 + 273.15)^2] [(T_1 + 273.15) + (T_2 + 273.15)]}{\left(\frac{1}{\varepsilon_1} + \frac{1}{\varepsilon_2} - 1\right)} \quad (3.5)$$

where σ is the Stefan-Boltzmann constant.

Notes:

- Temperatures are internally converted to kelvin.
- This coefficient is useful for estimating radiative exchange between layers in solar collectors, building envelopes, or other thermal systems.
- The formula accounts for the combined effect of both surfaces' emissivities.

Arguments:

- **T_p1 [float]:** Temperature of plate 1 in degrees Celsius.
- **epsilon_1 [float]:** Emissivity of plate 1 (dimensionless, between 0 and 1).
- **T_p2 [float]:** Temperature of plate 2 in degrees Celsius.
- **epsilon_2 [float]:** Emissivity of plate 2 (dimensionless, between 0 and 1).

3.6 conductive_heat_transfer_coefficient()

This function calculates the conductive heat transfer coefficient, which quantifies the rate of heat transfer through a material per unit area and temperature difference. It is used in steady-state heat conduction analysis.

Formula:

$$h = \frac{k}{L} \quad (3.6)$$

Arguments:

- **k [float]:** Thermal conductivity of the material in W/(m·K).
- **L [float]:** Thickness of the material in meters.

3.7 Rayleigh()

This function calculates the Rayleigh number, a dimensionless quantity that characterizes natural convection between two horizontal plates. It is used to determine the onset and intensity of convective heat transfer in fluid layers.

Formula:

$$Ra = \frac{g \cdot \beta \cdot |T_{p1} - T_{p2}| \cdot L_{\text{gap}}^3}{\nu \cdot \alpha} \quad (3.7)$$

where $\alpha = \frac{k}{\rho \cdot c}$ is the thermal diffusivity.

Arguments:

- **T_p1 [float]:** Temperature of the upper plate in degrees Celsius.
- **T_p2 [float]:** Temperature of the lower plate in degrees Celsius.
- **L_gap [float]:** Distance between the plates in meters.
- **beta [float, optional]:** Thermal expansion coefficient in 1/K (default is 3.4×10^{-3}).
- **visc [float, optional]:** Kinematic viscosity in m^2/s (default is 1.562×10^{-5}).
- **k [float, optional]:** Thermal conductivity in $\text{W}/(\text{m}\cdot\text{K})$ (default is 0.024).
- **density [float, optional]:** Fluid density in kg/m^3 (default is 1.184).
- **c [float, optional]:** Specific heat capacity in $\text{J}/(\text{kg}\cdot\text{K})$ (default is 1005).
- **g [float, optional]:** Gravitational acceleration in m/s^2 (default is 9.8).

3.8 Nusselt()

This function estimates the Nusselt number for natural convection between two inclined parallel plates. The Nusselt number characterizes the enhancement of heat transfer through convection relative to conduction.

Formula:

$$Nu = 1 + 1.44 \left(1 - \frac{1708}{Ra \cdot \cos(\theta)} \right) \left(1 - \frac{1708 \cdot \sin^{1.6}(1.6\theta)}{Ra \cdot \cos(\theta)} \right) \left[\left(\frac{Ra \cdot \cos(\theta)}{5830} \right)^{1/3} - 1 \right] \quad (3.8)$$

where Ra is the Rayleigh number and θ is the tilt angle in radians.

Arguments:

- **T_p1 [float]:** Temperature of the upper plate in degrees Celsius.
- **T_p2 [float]:** Temperature of the lower plate in degrees Celsius.
- **L_gap [float]:** Distance between the plates in meters.
- **tilt [float, optional]:** Tilt angle of the plates in degrees (default is 0°).

3.9 Reynolds()

This function calculates the Reynolds number for internal flow in a circular tube. The Reynolds number is a dimensionless quantity used to predict flow regimes, such as laminar or turbulent flow.

Formula:

$$Re = \frac{4 \cdot \dot{m}}{\rho \cdot \pi \cdot D \cdot \nu} \quad (3.9)$$

where $\dot{m}$ is the mass flow rate, ρ is the fluid density, D is the tube diameter, and ν is the kinematic viscosity.

Arguments:

- **m_dot [float]:** Mass flow rate in kg/s .

- **D_tube [float]:** Inner diameter of the tube in meters.
- **density [float, optional]:** Fluid density in kg/m³ (default is 1000).
- **visc [float, optional]:** Kinematic viscosity in m²/s (default is 1.562×10^{-5}).

3.10 Prandlt()

This function calculates the Prandtl number, a dimensionless quantity that relates the momentum diffusivity (viscosity) to the thermal diffusivity of a fluid. It is used in heat transfer and fluid flow calculations.

Formula:

$$Pr = \frac{\nu}{\alpha} \quad (3.10)$$

where ν is the kinematic viscosity and $\alpha = \frac{k}{\rho \cdot c}$ is the thermal diffusivity.

Arguments:

- **visc [float, optional]:** Kinematic viscosity in m²/s (default is 8.927×10^{-7}).
- **k [float, optional]:** Thermal conductivity in W/(m·K) (default is 0.6071).
- **density [float, optional]:** Fluid density in kg/m³ (default is 1000).
- **c [float, optional]:** Specific heat capacity in J/(kg·K) (default is 4200).

3.11 fluid_conductive_heat_transfer()

This function calculates the convective heat transfer coefficient for fluid flow inside a circular tube. It uses different correlations depending on the flow regime: conduction for zero flow, a constant Nusselt number for laminar flow, and the Dittus-Boelter equation for turbulent flow.

Formulas:

- **No flow:**

$$h = \frac{2k_f}{D} \quad (3.11)$$

- **Laminar flow ($Re < 2300$):**

$$h = \frac{4.36k_f}{D} \quad (3.12)$$

- **Turbulent flow ($Re \geq 2300$):**

$$h = \frac{0.023 \cdot k_f \cdot Re^{0.8} \cdot Pr^{0.4}}{D} \quad (3.13)$$

Arguments:

- **m_dot [float]:** Mass flow rate in kg/s.
- **k_f [float]:** Thermal conductivity of the fluid in W/(m·K).
- **D_tube [float]:** Inner diameter of the tube in meters.

3.12 air_gap_convection()

This function calculates the effective convective heat transfer coefficient across an air gap between two surfaces. It uses the Nusselt number to account for natural convection effects.

Formula:

$$h = \frac{Nu \cdot k}{L_{\text{gap}}} \quad (3.14)$$

where Nu is the Nusselt number, k is the thermal conductivity of air, and L_{gap} is the distance between the surfaces.

Arguments:

- **T_p1 [float]:** Temperature of the first surface in degrees Celsius.
- **T_p2 [float]:** Temperature of the second surface in degrees Celsius.
- **L_gap [float]:** Distance between the surfaces in meters.
- **k [float]:** Thermal conductivity of the air in W/(m·K).

3.13 PVT_model()

This function simulates the thermal behavior of a photovoltaic-thermal (PVT) collector over a single time step. It calculates the temperatures of various components (glass, PV layer, absorber, and fluid) and the thermal power output of the system. As detailed in [4], the governing equations of the model, as a function of the surface temperatures of the glass cover, the Si cell, the thermal absorber, and the fluid, T_{glass} , T_{PV} , T_{a} and T_{f} , respectively, are

$$\begin{cases} T_{\text{glass}}(k+1) = C_1(k) T_{\text{glass}}(k) + C_2(k) T_{\text{PV}}(k) - D_1(k) \\ T_{\text{PV}}(k+1) = C_3(k) T_{\text{glass}}(k) + C_4(k) T_{\text{PV}}(k) + C_5(k) T_{\text{a}}(k) - D_2(k) \\ T_{\text{a}}(k+1) = C_6(k) T_{\text{PV}}(k) + C_7(k) T_{\text{a}}(k) + C_8(k) T_{\text{f}}(k) - D_3(k) \\ T_{\text{f}}(k+1) = C_9(k) T_{\text{a}}(k) + C_{10}(k) T_{\text{f}}(k) - D_4(k) \end{cases}, \quad (3.15)$$

where

$$C_1(k) = \frac{A_{\text{glass}} \Delta t}{m_{\text{glass}} c_{\text{glass}}} \left[\frac{m_{\text{glass}} c_{\text{glass}}}{A_{\text{glass}} \Delta t} - h_{\text{glass}}^{\text{conv}}(k) - h_{\text{glass}}^r(k) - h_{\text{gap}}(k) - h_{\text{glass-PV}}^r(k) \right], \quad (3.16)$$

$$C_2(k) = \frac{A_{\text{glass}} \Delta t}{m_{\text{glass}} c_{\text{glass}}} [h_{\text{gap}}(k) + h_{\text{glass-PV}}^r(k)], \quad (3.17)$$

$$C_3(k) = \frac{A_{\text{PV}} \Delta t}{m_{\text{PV}} c_{\text{PV}}} [h_{\text{gap}}(k) + h_{\text{glass-PV}}^r(k)], \quad (3.18)$$

$$C_4(k) = \frac{A_{\text{PV}} \Delta t}{m_{\text{PV}} c_{\text{PV}}} \left[\frac{m_{\text{PV}} c_{\text{PV}}}{A_{\text{PV}} \Delta t} - h_{\text{gap}}(k) - h_{\text{glass-PV}}^r(k) - h_{\text{PV-a}}^{\text{cond}}(k) \right], \quad (3.19)$$

$$C_5(k) = \frac{A_{\text{PV}} \Delta t}{m_{\text{PV}} c_{\text{PV}}} h_{\text{PV-a}}^{\text{cond}}(k), \quad (3.20)$$

$$C_6(k) = \frac{A_{\text{a}} \Delta t}{m_{\text{a}} c_{\text{a}}} h_{\text{PV-a}}^{\text{cond}}(k), \quad (3.21)$$

$$C_7(k) = \frac{\Delta t}{m_a c_a} \left[\frac{m_a c_a}{\Delta t} - h_{PV-a}^{\text{cond}}(k) A_a - h_{a-f}(k) A_t^{\text{cross}} - h_a^{\text{cond}}(k) A_a \right], \quad (3.22)$$

$$C_8(k) = \frac{\Delta t}{m_a c_a} h_{a-f}(k) A_t^{\text{cross}}, \quad (3.23)$$

$$C_9(k) = \frac{A_t^{\text{cross}} \Delta t}{m_f c_f} h_{a-f}(k), \quad (3.24)$$

$$C_{10}(k) = \frac{\Delta t}{m_f c_f} \left[\frac{m_f c_f}{\Delta t} - h_{a-f}(k) A_t^{\text{surf}} - 2\dot{m}_f c_f \right], \quad (3.25)$$

$$D_1(k) = \frac{A_{\text{glass}} \Delta t}{m_{\text{glass}} c_{\text{glass}}} \left[h_{\text{glass}}^{\text{conv}} T_{\text{amb}}(k) + h_{\text{glass}}^r(k) T_{\text{sky}}(k) + \alpha_{\text{glass}} G(k) \right], \quad (3.26)$$

$$D_2(k) = \frac{A_{PV} \Delta t}{m_{PV} c_{PV}} \alpha_{PV} \tau_{\text{glass}} G(k) [1 - \eta_{PV}(k)], \quad (3.27)$$

$$D_3(k) = \frac{A_a \Delta t}{m_a c_a} h_a^{\text{cond}}(k) T_{\text{amb}}(k), \quad (3.28)$$

and

$$D_4(k) = 2 \frac{\Delta t}{m_f} \dot{m}_f T_f^{\text{in}}(k). \quad (3.29)$$

Main Processes:

- Convective and radiative heat exchange with the environment.
- Heat conduction through the PVT layers.
- Fluid heat exchange in the absorber tubes.
- PV electrical efficiency adjustment based on temperature.

Arguments:

- **T_amb [float]:** Ambient temperature in °C.
- **G [float]:** Solar irradiance in W/m².
- **v [float]:** Wind speed in m/s.
- **T_f.in [float]:** Inlet fluid temperature in °C.
- **T_glass_0, T_PV_0, T_a_0, T_f_0 [float]:** Initial temperatures of the glass, PV, absorber, and fluid layers in °C.
- **n_STC [float]:** Nominal PV efficiency at standard test conditions.
- **N_tubes [int]:** Number of absorber tubes.
- **D_tube [float]:** Diameter of the absorber tubes in meters.
- **n_HE [float, optional]:** Heat exchanger efficiency (default is 0.8).
- **A_collector, A_PV [float, optional]:** Collector and PV surface areas in m².
- **m_f.dot [float, optional]:** Fluid mass flow rate in kg/s.
- **Len_collector [float, optional]:** Collector length in meters.
- **c_f [float, optional]:** Specific heat of the fluid in J/(kg·K).
- **dt [float, optional]:** Time step in seconds.

3.14 Qdot_PVT()

This function simulates the thermal performance of a photovoltaic-thermal (PVT) collector system over a specified time period. It models the transient behaviour of the system using the 'PVT_model()' function and updates the thermal state of the PVT tank and its interaction with a thermal energy storage system (TESS) and a thermal network. Formula:

$$\dot{Q}_{\text{PVT, mod}}(k) = \eta_T N \dot{m}_f c_f [T_f^{\text{out}}(k) - T_f^{\text{in}}(k)] = 2\eta_T N \dot{m}_f c_f [T_f(k) - T_f^{\text{in}}(k)] , \quad (3.30)$$

Main Processes:

- Time-stepped simulation of the PVT collector using recirculation.
- Updates to the PVT tank temperature and heat exchange with the network and TESS.
- Calculation of thermal power delivered to the network and TESS.

Arguments:

- **PVT_active [bool]:** Whether the PVT system is active.
- **TESS_charge [bool]:** Whether the TESS is being charged.
- **T_amb [float]:** Ambient temperature in °C.
- **G [float]:** Solar irradiance in W/m².
- **v [float]:** Wind speed in m/s.
- **T_in_Network [float]:** Inlet temperature of the thermal network in °C.
- **T_glass_0, T_PV_0, T_a_0, T_f_0 [float]:** Initial temperatures of the glass, PV, absorber, and fluid layers in °C.
- **T_tank_PVT [float]:** Initial temperature of the PVT tank in °C.
- **T0_TESS [float]:** Initial temperature of the TESS in °C.
- **n_STC [float, optional]:** Nominal PV efficiency at standard test conditions (default is 0.184).
- **N_tubes [int, optional]:** Number of absorber tubes (default is 0).
- **D_tube [float, optional]:** Diameter of the absorber tubes in meters (default is 0.009).
- **n_HE [float, optional]:** Heat exchanger efficiency (default is 0.8).
- **A_collector, A_PV [float, optional]:** Collector and PV surface areas in m².
- **m_f_dot [float, optional]:** Fluid mass flow rate in kg/s.
- **Len_collector [float, optional]:** Collector length in meters.
- **t_end [int, optional]:** Simulation duration in seconds (default is 900 s).
- **n_modules [int, optional]:** Number of PVT modules (default is 1).
- **recirculation [bool, optional]:** Whether to simulate with recirculation (default is True).

3.15 T_fluid_out()

This function estimates the outlet temperature of the fluid in a simplified heat exchange model. It assumes a linear relationship between the inlet and fluid temperatures.

Formula:

$$T_{\text{out}} = \frac{T_f - 0.5 \cdot T_{\text{in}}}{0.5} \quad (3.31)$$

Arguments:

- **T_in [float]:** Inlet temperature in degrees Celsius.
- **T_f [float]:** Fluid temperature in degrees Celsius.

3.16 PV_efficiency()

This function calculates the temperature-adjusted efficiency of a photovoltaic (PV) module. The efficiency decreases linearly with increasing temperature above a reference value.

Formula:

$$\eta = \eta_{\text{STC}} [1 - \beta_{\text{PV}} \cdot (T_{\text{PV}} - T_{\text{ref}})] \quad (3.32)$$

Arguments:

- **T_PV [float]:** PV cell temperature in degrees Celsius.
- **n_STC [float]:** Nominal efficiency at standard test conditions (STC).
- **beta_PV [float, optional]:** Temperature coefficient in 1/°C (default is -0.003).
- **T_ref [float, optional]:** Reference temperature in degrees Celsius (default is 25°C).

3.17 PV_out()

This function calculates the electrical power output of a photovoltaic (PV) module based on its temperature, irradiance, and area. It adjusts the nominal efficiency using the temperature-dependent model and returns the output in either kilowatts or watts.

Formula:

$$P_{\text{PV}} = \eta(T_{\text{PV}}) \cdot G \cdot A_{\text{PV}} \quad (3.33)$$

where $\eta(T_{\text{PV}})$ is the temperature-adjusted efficiency, G is the solar irradiance in W/m², and A_{PV} is the PV area in m².

Arguments:

- **T_PV [float]:** PV cell temperature in degrees Celsius.
- **G [float]:** Solar irradiance in W/m².
- **A_PV [float, optional]:** PV surface area in m² (default is 60 · 0.156²).
- **n_STC [float, optional]:** Nominal efficiency at standard test conditions (default is 0.184).
- **kW [bool, optional]:** If True, returns output in kilowatts (default); otherwise in watts.

3.18 Update_PVT_tank()

This function updates the temperature of the PVT tank based on energy balances involving the PVT system, the thermal network, and the thermal energy storage system (TESS). It uses simplified heat exchanger models to compute energy transfers.

Formula:

$$T_{\text{tank}} = T_0 + \frac{\Delta t}{m \cdot c} \left(Q_{\text{PVT}} - Q_{\text{tank}} - \frac{Q_{\text{TESS}}}{\eta_{\text{TESS}}} \right) \quad (3.34)$$

Arguments:

- **T_in_PVT [float]:** Inlet temperature to the PVT system in °C.
- **T_in_Network [float]:** Inlet temperature from the thermal network in °C.
- **T0 [float]:** Initial tank temperature in °C.
- **T0_TESS [float]:** Initial TESS temperature in °C.
- **PVT_active [bool, optional]:** Whether the PVT system is active (default is True).
- **TESS_charge [bool, optional]:** Whether the TESS is being charged (default is False).
- **mdot [float, optional]:** Mass flow rate in kg/s (default is 0.029085).
- **m [float, optional]:** Mass of the tank contents in kg (default is 200).
- **c [float, optional]:** Specific heat capacity in J/(kg·K) (default is 4200).
- **efficiency_TESS [float, optional]:** Efficiency of the TESS heat exchanger (default is 0.8).
- **dt [float, optional]:** Time step in seconds (default is 1).

3.19 Heat_exchanger()

This function models a simplified heat exchanger using either a liquid-to-liquid (L2L) or a thermal energy storage system (TESS) mode. It calculates the outlet temperature and heat transfer rate based on mass flow rates, specific heat capacities, and efficiency.

Modes:

- **L2L:** Heat is transferred from stream 2 to stream 1.
- **TESS:** Heat is transferred from stream 1 to a thermal storage medium.

Formulas:

- **L2L mode:**

$$\dot{Q}_1 = \eta \cdot \dot{m}_2 \cdot c_2 \cdot (T_{2,\text{in}} - T_{1,\text{in}}) \quad (3.35)$$

$$T_{1,\text{out}} = T_{1,\text{in}} + \frac{\dot{Q}_1}{\dot{m}_1 \cdot c_1} \quad (3.36)$$

- **TESS mode:**

$$\dot{Q}_2 = \eta \cdot \dot{m}_1 \cdot c_1 \cdot (T_{1,\text{in}} - T_{2,\text{in}}) \quad (3.37)$$

Arguments:

- **T1_in [float]:** Inlet temperature of stream 1 in °C.
- **T2_in [float]:** Inlet temperature of stream 2 in °C.
- **mdot1 [float, optional]:** Mass flow rate of stream 1 in kg/s (default is 0.22).
- **mdot2 [float, optional]:** Mass flow rate of stream 2 in kg/s (default is 0.029085).
- **c1, c2 [float, optional]:** Specific heat capacities in J/(kg·K) (default is 4200).
- **efficiency [float, optional]:** Heat exchanger efficiency (default is 0.8).
- **mode [str, optional]:** Operation mode: "L2L" or "TESS" (default is "L2L").

3.20 Qdot_SolarCollector()

This function calculates the thermal power output of a simple solar collector based on its surface area, efficiency, and the incident solar irradiance. The output is averaged over a given time step. Formula:

$$\dot{Q}_{SC} = \begin{cases} \frac{A \cdot \eta_{SC} \cdot G}{\Delta t}, & \text{if active} \\ 0, & \text{otherwise} \end{cases} \quad (3.38)$$

Arguments:

- **active [bool]:** Whether the solar collector is active.
- **G [float]:** Solar irradiance in W/m².
- **A [float, optional]:** Collector surface area in m² (default is 6).
- **SC_eff [float, optional]:** Collector efficiency (default is 0.45).
- **dt [float, optional]:** Time step in seconds (default is 3600 s).

3.21 update_TESS()

This function updates the temperature of a Thermal Energy Storage System (TESS) based on heat exchange with a solar collector, a heat pump, and heat losses to the surrounding soil. It uses an energy balance approach to simulate charging and discharging behavior. Formula:

$$T_{\text{new}} = T_0 + \frac{\Delta t}{m \cdot c} \left(Q_{SC} \cdot \eta + Q_{HP} \cdot \eta - Q_{SD} - \frac{Q_{TESS}}{\eta} \right) \quad (3.39)$$

Arguments:

- **active [bool]:** Whether the TESS is active.
- **T_in [float]:** Inlet temperature to the TESS in Kelvin.
- **T_0 [float]:** Current temperature of the TESS in Kelvin.
- **T_soil [list of float]:** Soil temperature profile in Kelvin.

- **Qdot_SC [float, optional]:** Heat input from the solar collector in watts (default is 0).
- **Qdot_HP [float, optional]:** Heat input from the heat pump in watts (default is 0).
- **Tmax [float, optional]:** Maximum operating temperature in Kelvin (default is 368 K).
- **Tmin [float, optional]:** Minimum operating temperature in Kelvin (default is 323 K).
- **mdot [float, optional]:** Mass flow rate in kg/s (default is 0.1).
- **T_network [float, optional]:** Network temperature in Kelvin (default is 323 K).
- **efficiency [float, optional]:** Efficiency of heat exchange (default is 0.8).
- **m [float, optional]:** Mass of the TESS contents in kg (default is 4000).
- **c [float, optional]:** Specific heat capacity in J/(kg·K) (default is 4200).
- **dt [float, optional]:** Time step in seconds (default is 900 s).

3.22 Qdot_SD_TESS2Soil()

This function estimates the heat loss from a Thermal Energy Storage System (TESS) to the surrounding soil. It considers heat transfer through the top, sides, and bottom surfaces of the TESS enclosure using a simplified conduction model.

Formula:

$$\dot{Q}_{SD} = \dot{Q}_{top} + \dot{Q}_{sides} + \dot{Q}_{bottom} \quad (3.40)$$

Details on the discretization of the TESS walls can be found in [4], but follow the formulas below:

$$\dot{Q}_{top}^{SD} = U_{TESS} A_{top} [T_{TESS}(k) - T_s(y = y_0, k)] , \quad (3.41)$$

$$\dot{Q}_{sides}^{SD} = \sum_{i=1}^n U_{TESS} A_{\Delta y} [T_{TESS}(k) - T_s(y_i, k)] , \quad (3.42)$$

$$\dot{Q}_{bottom}^{SD} = U_{TESS} A_{bottom} [T_{TESS}(k) - T_s(y = y_f, k)] . \quad (3.43)$$

Arguments:

- **T_soil [list of float]:** Soil temperature profile in °C.
- **T_TESS [float]:** Temperature of the TESS in °C.
- **m [float, optional]:** Mass of the TESS in kg (default is 4000).
- **dy [float, optional]:** Vertical discretization step in meters (default is 0.1).
- **l_TESS, w_TESS, h_TESS [float, optional]:** Length, width, and height of the TESS in meters.
- **t_TESS [float, optional]:** Thickness of the insulation layer in meters (default is 0.4).
- **k_styro [float, optional]:** Thermal conductivity of the insulation material in W/(m·K) (default is 0.033).
- **h_styro [float, optional]:** Convective heat transfer coefficient (unused, default is 0).

3.23 HP_Power_0()

This function calculates the electrical and thermal power output of a heat pump operating at a fixed coefficient of performance (COP). It assumes a constant input power and returns zero output if the heat pump is inactive.

Formulas:

$$P_{\text{el}} = \frac{P_{\text{in}}}{1000} \quad (3.44)$$

$$P_{\text{th}} = P_{\text{in}} \cdot \text{COP} \quad (3.45)$$

Arguments:

- **active [bool]**: Whether the heat pump is active.
- **P_in [float, optional]**: Electrical input power in watts (default is 2700 W).
- **COP [float, optional]**: Coefficient of performance (default is 4.1).

3.24 HP_Power()

This function calculates the electrical and thermal power output of a heat pump based on ambient and return temperatures, following the approximation proposed in [6]. It uses an exponential model for the coefficient of performance (COP) and limits the electrical power to a maximum threshold.

Formulas:

$$\text{COP} = 7.90471 \cdot e^{-0.024 \cdot (T_{\text{ret}} - T_{\text{amb}})} \quad (3.46)$$

$$P_{\text{el}} = \min \left(\frac{\dot{m} \cdot c \cdot (T_{\text{sup}} - T_{\text{ret}})}{1000 \cdot \text{COP}}, P_{\text{max}} \right) \quad (3.47)$$

$$P_{\text{th}} = \begin{cases} \eta_{\text{HE}} \cdot \dot{m} \cdot c \cdot (T_{\text{sup}} - T_{\text{ret}}), & \text{if } P_{\text{el}} < P_{\text{max}} \\ P_{\text{max}} \cdot \text{COP} \cdot 1000, & \text{otherwise} \end{cases} \quad (3.48)$$

Arguments:

- **active [bool]**: Whether the heat pump is active.
- **T_amb [float]**: Ambient temperature in Kelvin.
- **T_ret [float]**: Return temperature from the system in Kelvin.
- **m_dot [float, optional]**: Mass flow rate in kg/s (default is 0.22).
- **HE_eff [float, optional]**: Heat exchanger efficiency (default is 0.8).
- **T_sup [float, optional]**: Supply temperature in Kelvin (default is 326 K).
- **P_max [float, optional]**: Maximum electrical power in kW (default is 6).
- **c [float, optional]**: Specific heat capacity in J/(kg·K) (default is 4200).

3.25 BESS_perm_min()

This function calculates the minimum permissible power exchange (charging or discharging) for a Battery Energy Storage System (BESS) based on its current state of charge (SoC). The result is clipped to the maximum allowable power limits.

Formula:

$$P_{\min} = \min \left(\frac{C_{\text{BESS}} \cdot (\text{SoC} - \text{SoC}_{\max})}{\Delta t}, -P_{\text{BESS},\max} \right) \quad (3.49)$$

Arguments:

- **SoC [float]:** Current state of charge (0 to 1).
- **Capacity_BESS [float, optional]:** Total capacity of the BESS in kWh (default is 3.36).
- **SoCmax [float, optional]:** Maximum allowable state of charge (default is 0.9).
- **P_BESS_max [float, optional]:** Maximum power exchange in kW (default is 1.28).
- **dt [float, optional]:** Time step in hours (default is 0.25).

3.26 BESS_perm_max()

This function calculates the maximum permissible power exchange (charging or discharging) for a Battery Energy Storage System (BESS) based on its current state of charge (SoC). The result is clipped to the maximum allowable power limits.

Formula:

$$P_{\max} = \max \left(\frac{C_{\text{BESS}} \cdot (\text{SoC} - \text{SoC}_{\min})}{\Delta t}, P_{\text{BESS},\max} \right) \quad (3.50)$$

Arguments:

- **SoC [float]:** Current state of charge (0 to 1).
- **Capacity_BESS [float, optional]:** Total capacity of the BESS in kWh (default is 3.36).
- **SoCmin [float, optional]:** Minimum allowable state of charge (default is 0.2).
- **P_BESS_max [float, optional]:** Maximum power exchange in kW (default is 1.28).
- **dt [float, optional]:** Time step in hours (default is 0.25).

3.27 update_BESS()

This function simulates the operation of a Battery Energy Storage System (BESS) over a time step. It determines whether the battery should charge, discharge, or remain idle based on the current state of charge (SoC), power demand, and PV generation. It also applies efficiency losses and limits based on system constraints.

Main Logic:

- If $\text{SoC} \geq \text{SoC}_{\max}$, the battery can only discharge.
- If $\text{SoC}_{\min} < \text{SoC} < \text{SoC}_{\max}$, the battery can charge or discharge.

- If $\text{SoC} \leq \text{SoC}_{\min}$, the battery can only charge.
- Power exchange is limited by `BESS_perm_min()` and `BESS_perm_max()`.

Arguments:

- **SoC_0 [float]:** Initial state of charge (0 to 1).
- **P_Load [float]:** Power demand in kW.
- **P_PV [float]:** PV power generation in kW.
- **SoCmax [float, optional]:** Maximum SoC (default is 0.9).
- **SoCmin [float, optional]:** Minimum SoC (default is 0.2).
- **P_BESS_max [float, optional]:** Maximum charge/discharge power in kW (default is 1.28).
- **P_Grid_max [float, optional]:** Maximum grid power in kW (default is 0).
- **Capacity_BESS [float, optional]:** Battery capacity in kWh (default is 3.36).
- **charge_efficiency [float, optional]:** Charging efficiency (default is 0.943).
- **discharge_efficiency [float, optional]:** Discharging efficiency (default is 0.943).
- **P_SD [float, optional]:** Self-discharge rate (default is 0).
- **dt [float, optional]:** Time step in hours (default is 0.25).

3.28 heat_loss()

This function calculates the heat loss (or gain) through a surface based on the overall heat transfer coefficient, surface area, and the temperature difference between the inside and outside environments.

Formula:

$$\dot{Q} = U \cdot A \cdot (T_{\text{out}} - T_{\text{in}}) \quad (3.51)$$

Arguments:

- **U [float]:** Overall heat transfer coefficient in $\text{W}/(\text{m}^2 \cdot \text{K})$.
- **Area [float]:** Surface area in m^2 .
- **T_in [float]:** Indoor temperature in $^{\circ}\text{C}$.
- **T_out [float]:** Outdoor temperature in $^{\circ}\text{C}$.

3.29 new_house_Temperature()

This function calculates the updated indoor temperature of a house based on thermal energy inputs and losses over a time step. It uses a lumped thermal mass model.

Formula:

$$T_{\text{new}} = T_0 + \frac{\Delta t}{mc_T} (Q_{\text{Losses}} + Q_{\text{HP}} + Q_{\text{TESS}} + Q_{\text{SC}} + Q_{\text{Boiler}}) \quad (3.52)$$

Arguments:

- **T_0 [float]:** Initial indoor temperature in °C.
- **Qdot_Losses [float]:** Heat losses from the house in watts.
- **mc_T [float]:** Thermal mass of the house (mass × specific heat) in J/K.
- **Qdot_HP [float, optional]:** Heat input from the heat pump in watts (default is 0).
- **Qdot_TESS [float, optional]:** Heat input from the thermal energy storage system in watts (default is 0).
- **Qdot_SC [float, optional]:** Heat input from the solar collector in watts (default is 0).
- **Qdot_Boiler [float, optional]:** Heat input from a boiler in watts (default is 0).
- **dt [float, optional]:** Time step in seconds (default is 900 s).

3.30 R_convective()

This function calculates the convective thermal resistance of a fluid flowing through a cylindrical tube. It uses the convective heat transfer coefficient derived from the Dittus-Boelter or laminar flow correlations.

Formula:

$$R_{\text{conv}} = \frac{1}{\pi \cdot D \cdot L \cdot h} \quad (3.53)$$

where:

- D is the inner diameter of the tube [m],
- L is the length of the tube [m],
- h is the convective heat transfer coefficient [W/m²·K], computed via `fluid_conductive_heat_transfer()`.

Arguments:

- **m_dot [float]:** Mass flow rate of the fluid in kg/s.
- **k_f [float, optional]:** Thermal conductivity of the fluid in W/(m·K) (default is 0.6071).
- **D_tube [float, optional]:** Inner diameter of the tube in meters (default is 13 mm).
- **L_tube [float, optional]:** Length of the tube in meters (default is 1 m).

3.31 R_conductive()

This function calculates the conductive thermal resistance of a cylindrical pipe wall using the logarithmic mean radius method.

Formula:

$$R_{\text{cond}} = \frac{\ln(D_{\text{out}}/D_{\text{in}})}{2\pi Lk} \quad (3.54)$$

where:

- D_{in} is the inner diameter of the pipe [m],
- D_{out} is the outer diameter of the pipe [m],
- L is the length of the pipe [m],
- k is the thermal conductivity of the pipe material [W/(m·K)].

Arguments:

- **D_in [float]:** Inner diameter of the pipe in meters.
- **D_out [float]:** Outer diameter of the pipe in meters.
- **k_tube [float]:** Thermal conductivity of the pipe material in W/(m·K).
- **L_tube [float, optional]:** Length of the pipe in meters (default is 1 m).

3.32 Celcius2Fahrenheit()

This function converts a temperature from degrees Celsius to degrees Fahrenheit. It is useful for applications where temperature data needs to be presented or processed in the Fahrenheit scale.

Formula:

$$T_F = T_C \cdot \frac{9}{5} + 32 \quad (3.55)$$

Arguments:

- **T_C [float]:** Temperature in degrees Celsius.

3.33 Infiltration_Airflow()

This function estimates the infiltration airflow rate (in m³/s) into a building due to temperature differences and wind effects. It uses a simplified empirical model based on the ASHRAE Handbook of Fundamentals [7], converting metric units to imperial units internally for compatibility with the formula.

Formula:

$$Q = \begin{cases} A_{\text{exposed}} \cdot A_{\text{unit}} \cdot \sqrt{C_{\text{stack}} \cdot |\Delta T| + C_{\text{wind}} \cdot U^2} \cdot 0.47194745/1000, & \text{if } \Delta T > 0 \\ 0, & \text{otherwise} \end{cases} \quad (3.56)$$

where:

- U is converted from m/s to mph: $U \leftarrow U \cdot 2.23694$

- A_{exposed} is converted from m^2 to ft^2 : $A_{\text{exposed}} \leftarrow A_{\text{exposed}} \cdot 10.7639$
- $\Delta T = T_{\text{in}}^{\circ F} - T_{\text{out}}^{\circ F}$
- $T^{\circ F} = T^{\circ C} \cdot \frac{9}{5} + 32$

Arguments:

- **T_in [float]:** Indoor temperature in degrees Celsius.
- **T_out [float]:** Outdoor temperature in degrees Celsius.
- **U [float, optional]:** Wind speed in meters per second (default is 1 m/s).
- **A_exposed [float, optional]:** Exposed surface area of the building in square meters (default is 119.6 m^2).
- **C_stack [float, optional]:** Stack coefficient (default is 0.015).
- **C_wind [float, optional]:** Wind coefficient (default is 0.0012).

3.34 Ventilation_Airflow()

This function estimates the airflow rate (in m^3/s) required for mechanical ventilation in a residential building. It is based on ASHRAE Handbook of Fundamentals [7] and considers the conditioned floor area and number of bedrooms.

Formula:

$$Q = \frac{0.47194745}{1000} [0.03 \cdot A_{\text{conditioned}} + 7.5 \cdot (1 + N_{\text{bedrooms}})] \quad (3.57)$$

where:

- $A_{\text{conditioned}}$ is converted from m^2 to ft^2 : $A_{\text{conditioned}} \leftarrow A_{\text{conditioned}} \cdot 10.7639$

Arguments:

- **A_conditioned [float, optional]:** Conditioned floor area in square meters (default is 120 m^2).
- **N_bedrooms [int, optional]:** Number of bedrooms in the dwelling (default is 1).

3.35 Infiltration_Ventilation_Heat_loss()

This function estimates the total heat loss (in watts) due to both infiltration and mechanical ventilation in a building. It combines airflow rates from infiltration and ventilation with the temperature difference between indoor and outdoor air.

Formula:

$$\dot{Q} = -c_{\text{air}} \cdot \rho_{\text{air}} \cdot (Q_{\text{inf}} + Q_{\text{vent}}) \cdot (T_{\text{in}} - T_{\text{out}}) \quad (3.58)$$

where:

- Q_{inf} is the infiltration airflow rate from `Infiltration_Airflow()`.
- Q_{vent} is the ventilation airflow rate from `Ventilation_Airflow()`.

Arguments:

- **T_in [float]:** Indoor temperature in degrees Celsius.
- **T_out [float]:** Outdoor temperature in degrees Celsius.
- **U [float, optional]:** Wind speed in meters per second (default is 1 m/s).
- **A_exposed [float, optional]:** Exposed surface area in square meters (default is 119.6 m²).
- **C_stack [float, optional]:** Stack coefficient (default is 0.015).
- **C_wind [float, optional]:** Wind coefficient (default is 0.0012).
- **A_conditioned [float, optional]:** Conditioned floor area in square meters (default is 120 m²).
- **N_bedrooms [int, optional]:** Number of bedrooms (default is 1).
- **c_air [float, optional]:** Specific heat capacity of air in J/kg·K (default is 1012).
- **density_air [float, optional]:** Air density in kg/m³ (default is 1.293).

3.36 pipe_losses()

This function estimates the thermal power loss (in watts) through a pipe system due to conduction and convection. It calculates the total thermal resistance of the pipe and uses it to compute the heat loss based on the temperature difference between the fluid and the environment.

Formula:

$$\dot{Q}_{\text{loss}} = \frac{T_f - T_{\text{out}}}{R_{\text{total}}} \quad (3.59)$$

where:

$$R_{\text{total}} = R_{\text{convective}} + \sum_{i=0}^{n-1} R_{\text{conductive},i} \quad (3.60)$$

Arguments:

- **T_f [float]:** Fluid temperature in degrees Celsius.
- **T_out [float]:** External temperature in degrees Celsius.
- **m_dot [float]:** Mass flow rate of the fluid in kg/s.
- **diameters [list of float, optional]:** List of pipe diameters in millimeters, from inner to outer layers (default is [13, 15, 40]).
- **k_f [float, optional]:** Thermal conductivity of the fluid in W/m·K (default is 0.6071).
- **k_tubes [list of float, optional]:** List of thermal conductivities for each pipe layer in W/m·K (default is [398, 0.038]).
- **L_tube [float, optional]:** Length of the pipe in meters (default is 1 m).

3.37 House_type_adjust()

This function returns adjustment factors for different types of residential buildings. These factors are used to scale thermal properties such as window area, wall area, roof area, and internal volume based on the house type.

Adjustment Factors:

- **Window adjustment factor**
- **Wall adjustment factor**
- **Roof adjustment factor**
- **Volume adjustment factor**

House Types and Corresponding Factors:

- **'apartment'**: [1, 1, 1, 1]
- **'studio'**: [0.5, 0.663, 0.467, 0.467]
- **'stand_alone'**: [1.5, 2.04, 1, 2]

Arguments:

- **House_type [str, optional]**: Type of house. Options are 'apartment', 'studio', or 'stand_alone' (default is 'apartment').

3.38 Windows_losses_coefficient()

This function calculates the overall heat transfer coefficient (U-value) for windows based on the glazing type and thermal resistances of the window layers. It accounts for convective and conductive heat transfer through the window assembly.

Formula:

$$U = \left(\sum R_i \right)^{-1} \quad (3.61)$$

where the total thermal resistance $\sum R_i$ depends on the glazing type:

- **Single glazing:**

$$R = \frac{1}{h_{\text{air,window}}} + \frac{L_{\text{window}}}{k_{\text{window}}} + \frac{1}{h_{\text{window,atm}}}$$

- **Double glazing:**

$$R = \frac{1}{h_{\text{air,window}}} + \frac{L_{\text{window}}}{k_{\text{window}}} + \frac{L_{\text{cavity}}}{k_{\text{air}}} + \frac{L_{\text{window}}}{k_{\text{window}}} + \frac{1}{h_{\text{window,atm}}}$$

- **Triple glazing:**

$$R = \frac{1}{h_{\text{air,window}}} + 3 \cdot \frac{L_{\text{window}}}{k_{\text{window}}} + 2 \cdot \frac{L_{\text{cavity}}}{k_{\text{air}}} + \frac{1}{h_{\text{window,atm}}}$$

Arguments:

- **glazing_type** [str]: Type of glazing. Options are 'single', 'double', or 'triple'.
- **h_air_window** [float, optional]: Convective heat transfer coefficient on the indoor side (default is 25 W/m²·K).
- **LWindow** [float, optional]: Thickness of a single glass pane in meters (default is 0.004 m).
- **kWindow** [float, optional]: Thermal conductivity of the glass in W/m·K (default is 0.8).
- **LCavity** [float, optional]: Thickness of the air cavity between panes in meters (default is 0.014 m).
- **kAir** [float, optional]: Thermal conductivity of air in W/m·K (default is 0.0257).
- **h_window_atm** [float, optional]: Convective heat transfer coefficient on the outdoor side (default is 32 W/m²·K).

3.39 Roof_losses_coefficient()

This function calculates the overall heat transfer coefficient (U-value) for a roof assembly. It considers convective and conductive heat transfer through the roof layers and is used to estimate thermal losses through the roof.

Formula:

$$U = \left(\frac{1}{h_{\text{air,roof}}} + \frac{L_{\text{roof}}}{k_{\text{roof}}} + \frac{1}{h_{\text{roof,atm}}} \right)^{-1} \quad (3.62)$$

Arguments:

- **LRoof** [float, optional]: Thickness of the roof in meters (default is 0.2 m).
- **kRoof** [float, optional]: Thermal conductivity of the roof material in W/m·K (default is 0.04).
- **h_air_roof** [float, optional]: Convective heat transfer coefficient on the indoor side in W/m²·K (default is 12).
- **h_roof_atm** [float, optional]: Convective heat transfer coefficient on the outdoor side in W/m²·K (default is 38).

3.40 Wall_losses_coefficient()

This function calculates the overall heat transfer coefficient (U-value) for a wall assembly. It accounts for the presence of cavity insulation and external wall covering, as well as convective and conductive heat transfer through the wall layers.

Formula:

$$U = \left(\frac{1}{h_{\text{air,wall}}} + \frac{L_{\text{wall}}}{k_{\text{wall}}} + R_{\text{cavity}} + R_{\text{cover}} + \frac{1}{h_{\text{wall,atm}}} \right)^{-1} \quad (3.63)$$

where:

- $R_{\text{cavity}} = \frac{L_{\text{cavity}}}{k_{\text{cavity}}}$ if **Cavity_Filling** is **True**, otherwise 0

- $R_{\text{cover}} = \frac{L_{\text{external}}}{k_{\text{external}}}$ if `Wall_Cover` is `True`, otherwise 0

Arguments:

- **Cavity_Filling** [`bool`, `optional`]: Whether the wall cavity is filled with insulation (default is `False`).
- **Cavity_type** [`str`, `optional`]: Type of cavity material, either `'air'` (default, $k = 0.0257$) or `'EPS'` ($k = 0.035$).
- **Wall_Cover** [`bool`, `optional`]: Whether the wall has an external insulating cover (default is `False`).
- **LWall** [`float`, `optional`]: Thickness of the main wall layer in meters (default is 0.25 m).
- **kWall** [`float`, `optional`]: Thermal conductivity of the main wall material in W/m·K (default is 0.14).
- **LCavity** [`float`, `optional`]: Thickness of the cavity layer in meters (default is 0.05 m).
- **kCavity** [`float`, `optional`]: Thermal conductivity of the cavity material in W/m·K (default is 0.0257).
- **LExternal** [`float`, `optional`]: Thickness of the external cover in meters (default is 0.1 m).
- **kExternal** [`float`, `optional`]: Thermal conductivity of the external cover in W/m·K (default is 0.035).
- **h_air_wall** [`float`, `optional`]: Convective heat transfer coefficient on the indoor side in W/m²·K (default is 0.9).
- **h_wall_atm** [`float`, `optional`]: Convective heat transfer coefficient on the outdoor side in W/m²·K (default is 0.9).

3.41 House_Thermal_Losses()

This function estimates the total thermal losses (in watts) from a residential building due to transmission through windows, walls, and roof, as well as infiltration and ventilation. It also returns the total thermal capacity of the building envelope and indoor air.

Formula:

$$\dot{Q}_{\text{total}} = \dot{Q}_{\text{roof}} + \dot{Q}_{\text{windows}} + \dot{Q}_{\text{walls}} + \dot{Q}_{\text{inf+vent}} \quad (3.64)$$

Arguments:

- **T_0_in** [`float`]: Indoor air temperature in Kelvin.
- **T_amb** [`float`]: Outdoor ambient temperature in Kelvin.
- **U** [`float`, `optional`]: Wind speed in m/s (default is 1).
- **A_exposed** [`float`, `optional`]: Exposed surface area in m² (default is 119.6).
- **C_stack** [`float`, `optional`]: Stack coefficient (default is 0.015).
- **C_wind** [`float`, `optional`]: Wind coefficient (default is 0.0012).

- **A_conditioned [float, optional]:** Conditioned floor area in m² (default is 120).
- **N_bedrooms [int, optional]:** Number of bedrooms (default is 1).
- **c_air [float, optional]:** Specific heat capacity of air in J/kg·K (default is 1012).
- **density_air [float, optional]:** Air density in kg/m³ (default is 1.293).
- **House_type [str, optional]:** Type of house: 'apartment', 'studio', or 'stand_alone' (default is 'apartment').
- **glazing_type [str, optional]:** Type of window glazing: 'single', 'double', or 'triple' (default is 'double').
- **Cavity_Filling [bool, optional]:** Whether the wall cavity is filled with insulation (default is False).
- **Cavity_type [str, optional]:** Type of cavity material: 'air' or 'EPS' (default is 'air').
- **Wall_Cover [bool, optional]:** Whether the wall has an external insulating cover (default is False).
- **LRoof [float, optional]:** Roof thickness in meters (default is 0.2).
- **LCavity_wall [float, optional]:** Wall cavity thickness in meters (default is 0.05).

3.42 Thermal_Electrical_model()

This function simulates the thermal and electrical behavior of a hybrid energy system over a specified time horizon. It models the interactions between photovoltaic-thermal (PVT) collectors, a thermal energy storage system (TESS), a heat pump (HP), and a battery energy storage system (BESS), accounting for thermal losses, energy flows, and control logic.

Main Components:

- PVT collector model
- TESS charging/discharging
- Heat pump operation
- BESS operation
- Indoor temperature dynamics

Arguments:

- **Thermal_Components [list of bool]:** Flags to activate [PVT, TESS, HP].
- **T_in_0 [float]:** Initial indoor temperature (K).
- **T4 [float]:** Initial fluid temperature at the end of the loop (K).
- **T_TESS_0 [float]:** Initial TESS temperature (K).
- **T_tank_PVT [float]:** Initial PVT tank temperature (K).

- **T_PVT_layers** [list of float]: Initial temperatures of PVT layers (°C).
- **TESS_min_op_T** [float]: Minimum operating temperature for TESS (K).
- **TESS_max_op_T** [float]: Maximum operating temperature for TESS (K).
- **SoC_0_BEES** [float]: Initial state of charge of the battery (0–1).
- **T_amb** [array]: Ambient temperature profile (K).
- **T_set** [array]: Indoor temperature setpoint profile (K).
- **G** [array]: Solar irradiance profile (W/m²).
- **Tsoil** [array]: Soil temperature profile (K).
- **P_Load** [array]: Electrical load profile (kW).
- **t_final** [int]: Number of simulation steps.
- **n_modules** [int, optional]: Number of PVT modules (default is 0).
- **P_BEES_max** [float, optional]: Max battery power (kW, default is 1.28).
- **P_Grid_max** [float, optional]: Max grid power (kW, default is 0).
- **Capacity_BEES** [float, optional]: Battery capacity (kWh, default is 3.36).
- **PVT_2_TESS** [bool, optional]: Enable PVT to TESS charging (default is True).
- **HP_2_TESS** [bool, optional]: Enable HP to TESS charging (default is True).
- **Network_losses** [bool, optional]: Include network pipe losses (default is False).
- **m** [float, optional]: Mass of TESS (kg, default is 4000).
- **T_sup** [float, optional]: Supply temperature (K, default is 323).
- **T_TESS_max** [float, optional]: Max TESS temperature (K, default is 368).
- **m_dot** [float, optional]: Mass flow rate (kg/s, default is 0.22).
- **c_f** [float, optional]: Specific heat of fluid (J/kg·K, default is 4200).
- **A_PVT** [float, optional]: Area of one PVT module (m², default is 1.48).
- **dt** [float, optional]: Time step in seconds (default is 900).

3.43 Thermal_model()

This function simulates the thermal behavior of a hybrid energy system over a single time step. It models the thermal interactions between a photovoltaic-thermal (PVT) collector, a thermal energy storage system (TESS), and a heat pump (HP), including thermal losses and temperature propagation through the system.

Main Components:

- PVT collector model
- TESS heat exchange
- Heat pump contribution
- Indoor temperature update

Arguments:

- **Thermal_Components** [list of bool]: Flags to activate [PVT, TESS, HP].
- **T_in_0** [list of float]: Indoor temperature at current and previous time step (K).
- **T4** [float]: Fluid temperature at the end of the loop (K).
- **T_TESS_0** [float]: Initial TESS temperature (K).
- **T_amb** [float]: Ambient temperature (K).
- **T_set** [float]: Indoor temperature setpoint (K).
- **G** [float]: Solar irradiance (W/m^2).
- **Qdot_TESS** [float]: Heat flow from TESS (W).
- **Qdot_HP** [float]: Heat flow from HP (W).
- **n_modules** [int, optional]: Number of PVT modules (default is 5).
- **PVT_active** [bool, optional]: Whether PVT is active (default is False).
- **PVT_TESS_charge** [bool, optional]: Whether PVT is charging TESS (default is False).
- **T_tank_PVT** [float, optional]: PVT tank temperature ($^{\circ}\text{C}$, default is 50).
- **T_PVT_layers** [list of float, optional]: Temperatures of PVT layers ($^{\circ}\text{C}$, default is [25, 25, 25]).
- **TESS_min_op_T** [float, optional]: Minimum operating temperature for TESS (K, default is 328).
- **TESS_max_op_T** [float, optional]: Maximum operating temperature for TESS (K, default is 363).
- **Network_losses** [bool, optional]: Include network pipe losses (default is False).
- **m** [float, optional]: Mass of TESS (kg, default is 4000).
- **T_sup** [float, optional]: Supply temperature (K, default is 323).

- **T_TESS_max** [float, optional]: Maximum TESS temperature (K, default is 368).
- **m_dot** [float, optional]: Mass flow rate (kg/s, default is 0.22).
- **c_f** [float, optional]: Specific heat of fluid (J/kg·K, default is 4200).
- **dt** [float, optional]: Time step in seconds (default is 900).

Bibliography

- [1] J. Alpízar-Castillo, “Residential multi-carrier energy storage systems as potential flexibility providers in low-voltage networks: A new player has joined the game,” Ph.D. dissertation, Delft University of Technology, 2025. [Online]. Available: <https://repository.tudelft.nl/record/uuid:76075049-7656-4c49-b206-e742a65b6062>.
- [2] J. Alpízar-Castillo, “Model predictive control implementation for the ocean grazer wave energy converter with a port-hamiltonian model,” M.S. thesis, University of Groningen, Costa Rica Institute of Technology, 2018. [Online]. Available: <https://repositoriotec.tec.ac.cr/handle/2238/10439>.
- [3] J. Alpízar-Castillo, *CSV Python Tools*, https://github.com/jjac13/CSV_Python_Tools, 2022.
- [4] J. Alpízar-Castillo, L. M. Ramírez-Elizondo, and P. Bauer, “Modelling and evaluating different multi-carrier energy system configurations for a dutch house,” *Applied Energy*, vol. 364, p. 123 197, 2024, ISSN: 0306-2619. DOI: 10.1016/j.apenergy.2024.123197. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306261924005804>.
- [5] J. Alpízar-Castillo, *MCES Library*, https://github.com/jjac13/MCES_Library, 2024.
- [6] N. Damianakis, G. C. R. Mouli, and P. Bauer, “Risk-averse estimation of electric heat pump power consumption,” in *2023 IEEE 17th International Conference on Compatibility, Power Electronics and Power Engineering (CPE-POWERENG)*, 2023, pp. 1–6. DOI: 10.1109/CPE-POWERENG58103.2023.10227424. [Online]. Available: <https://repository.tudelft.nl/record/uuid:e5431373-7f01-4029-a467-d27e69ace94f>.
- [7] American Society of Heating, Refrigerating and Air-Conditioning Engineers, Inc. (ASHRAE), *2021 ASHRAE® Handbook - Fundamentals (SI Edition)*. American Society of Heating, Refrigerating and Air-Conditioning Engineers, Inc. (ASHRAE), 2021, ISBN: 978-1-947192-90-4.